QA WORKSPACE PRO

Software Requirements Specification

A unified QA productivity tool for Playwright automation testers

Document	SRS — QA Workspace Pro
Version	1.0 — Initial Draft
Date	March 2026
Prepared by	QA Trainee — VnV Batch Jan 2026
Status	Draft — For Review

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for QA Workspace Pro — a single-page web application designed to help automation testers plan, generate, track, and export all test artifacts without switching between tools.

This document is intended for: the developer building the tool, the tester using it daily, and any reviewer assessing its scope and feasibility.

1.2 Background & Problem Statement

The Problem

A QA engineer on a Playwright + TypeScript + Agile project currently uses 6+ separate tools for one sprint: JIRA for stories, Excel/Sheets for test cases, a text editor for Gherkin, browser DevTools for locators, Jenkins for CI results, and Word for defect reports. None of them talk to each other. Context-switching kills productivity and causes things to fall through the cracks.

QA Workspace Pro solves this by combining the most-used test engineering utilities into one coherent workspace, where every tab shares the same project context and outputs feed into each other.

1.3 Scope

QA Workspace Pro is a browser-based, single-page application. It covers:

- Test case creation, structuring, and status tracking
- Smoke and sanity checklist generation
- Coverage heatmap by feature area
- Boundary value analysis with automatic test case generation
- Equivalence class partitioning
- Playwright BDD spec and locator generation (Gherkin + step definitions)
- Defect report quality scoring and JIRA-ready output
- SQL schema-aware test data generation
- CI/CD test result analysis (Playwright JSON report parsing)
- Agile sprint test planning with RTM and DoD checklist
- One-click full test plan export

Out of scope for v1.0: real JIRA API integration, live Jenkins connection, user authentication, database persistence (local session only).

1.4 Definitions & Acronyms

Term	Meaning
BDD	Behaviour-Driven Development — writing tests as human-readable scenarios
TDD	Test-Driven Development — writing tests before code

POM	Page Object Model — design pattern for Playwright/Selenium automation
RTM	Requirement Traceability Matrix — maps requirements to test cases
DoD	Definition of Done — sprint completion checklist
STLC	Software Testing Life Cycle
Gherkin	Given/When/Then language used in Cucumber/Playwright BDD
EP	Equivalence Partitioning — grouping inputs into valid/invalid classes
BVA	Boundary Value Analysis — testing at min/max input limits
CI/CD	Continuous Integration / Continuous Deployment (Jenkins pipeline)

2. Overall Description

2.1 Product Overview

QA Workspace Pro is a 9-module tabbed workspace. All modules share a single project context header (Feature Name, Version, Tester, Sprint). Data flows between modules — boundary values auto-populate the case formatter; equivalence classes do the same; coverage heatmap reflects case completion status.

2.2 Architecture (Conceptual)

Single-Page Architecture
Frontend only — HTML + CSS + JavaScript (React or vanilla). No backend server required for v1.0. All state is held in memory during the session. Export generates a downloadable .txt or .docx file. No login, no database. Runs in any modern browser.

2.3 User Profile

User Type	Background	Primary Use
Junior QA / Trainee	Recently completed Playwright + JS/TS training	Plan tests, write BDD specs, generate test cases from scratch
Mid-level QA	1–3 years QA experience, knows JIRA and Agile	Speed up sprint planning, defect reporting, and CI result analysis
Team Lead / Test Manager	Reviews test plans and defect quality	Review exported test plan document, coverage heatmap

2.4 Assumptions & Constraints

- User has basic knowledge of Playwright, JIRA, and Agile
- The tool runs client-side only — no server, no cloud storage in v1.0
- All data is lost on browser refresh unless exported first
- Playwright BDD spec generation produces skeletons — user fills in actual locators
- SQL test data generation works with standard SQL CREATE TABLE syntax
- CI report parsing supports Playwright's JSON reporter format

3. Functional Requirements — Modules

Shared Project Context (all modules)
Every module reads from a persistent header: Feature/Project Name, Version, Sprint Number, Tester Name. This is set once and shared everywhere. Changing it updates all module outputs automatically.

Module 1 — Case Formatter

Purpose

Convert rough test notes (bullet points, stream-of-consciousness writing) into properly structured test cases with type, priority, expected result, and execution status.

Inputs

- Free-text test notes (multi-line, comma or newline separated)
- Test type selector: Functional / Negative / Boundary / Security / UI / Performance / Smoke / Regression
- Priority selector: High / Medium / Low

Outputs

- Structured test case table: ID, Title, Type, Priority, Expected Result, Status
- Status per case: Todo / Pass / Fail / Skip / Blocked
- Summary stats: total, pass, fail, skip counts

Behaviour Rules

1. Each line/comma item in notes becomes one test case
2. System infers expected result from keywords (e.g. 'invalid' → 'error message shown', 'valid' → 'success, redirect to dashboard')
3. Cases from Boundary Calculator and Equivalence Partitioner auto-append here via 'Send to Cases' action
4. All cases are included in the final export

Module 2 — Checklist Builder

Purpose

Generate smoke/sanity/regression checklists from a feature description so the tester never starts from a blank page before release.

Inputs

- Feature description (free text)
- Checklist type: Smoke / Sanity / Regression / Playwright Automation / API Test Points

Outputs

- Checklist with checkboxes, grouped by checklist type
- Progress tracker: X of Y done, percentage complete

Behaviour Rules

5. Each checklist type has a built-in template of 6–10 items that can be customised
6. User can add and remove individual items
7. Checked state is preserved within the session

Module 3 — Coverage Heatmap

Purpose

Visualise which feature areas have sufficient test coverage and which are under-tested, using a colour-coded grid.

Inputs

- Feature area names (user adds them manually, e.g. Login, Cart, API, Admin)

Outputs

- Grid of colour-coded cells: grey (0 cases) → light green (1-2) → medium green (3-5) → dark green (6+)
- Click a cell to increment its case count

Behaviour Rules

8. Coverage count per area is also derived from Case Formatter (cases tagged to that area increase its count automatically in v1.1)
9. Heatmap is included as a summary table in the exported test plan

Module 4 — Boundary Calculator

Purpose

Given a field's data type, minimum and maximum, automatically compute all 7 standard boundary test values and generate ready-to-use test cases.

Inputs

- Field name, data type (Integer / Float / String length / Date), min value, max value
- Optional: additional constraint description

Outputs

- 7 boundary values: below-min (invalid), min, min+1, midpoint, max-1, max, above-max (invalid)

- Each value labelled valid or invalid
- 'Send to Case Formatter' button populates cases automatically

Module 5 — Equivalence Partitioner

Purpose

Given a field's validation rules in plain English, generate valid, invalid, and edge-case equivalence partitions with example values and descriptions.

Inputs

- Field name, validation rules in plain text (e.g. '6–12 chars, alphanumeric, first char must be a letter')

Outputs

- Partition cards labelled: Valid / Invalid / Edge
- Each card shows: partition label, description, example value
- 'Send to Case Formatter' button auto-creates test cases for all partitions

Module 6 — Playwright Spec Builder

Purpose

The biggest time-saver for day-to-day automation work. Given a user story and acceptance criteria, generate a complete BDD skeleton: Gherkin feature file, TypeScript step definition stubs, Playwright locator suggestions, and a Page Object Model class stub — all in one go.

Inputs

- User story: 'As a [role] I want to [goal] so that [benefit]'
- Acceptance criteria (one per line)
- Locator hint: CSS selector or element description (optional)

Outputs

- Gherkin .feature file text (Feature, Scenario, Given/When/Then)
- TypeScript step definition skeleton (.ts) with matching step strings
- Suggested Playwright locators: role, text, CSS, XPath variants
- Page Object Model class stub with methods mapped to each step
- BDD tags auto-added: @smoke, @regression per priority
- Downloadable as .feature and .ts files

Behaviour Rules

10. Each acceptance criterion line becomes one Scenario
11. Step definitions are generated as empty async functions with correct import statements
12. POM class includes constructor with page param, and one method stub per interaction step

13. Locator suggestions include: getByRole, getByText, locator('[data-testid]'), XPath fallback

Why this matters

In a real project you will get a JIRA story and be asked to automate it. Right now that means manually writing the feature file, then opening a .ts file for step defs, then figuring out locators, then sketching the POM — four separate steps across four files. This module collapses all of them into one 30-second operation.

Module 7 — Defect DNA Analyzer

Purpose

Score a rough defect description against 8 quality dimensions and auto-generate a properly formatted JIRA-ready bug report. Prevents the most common source of wasted stand-up time: bad defect reports.

Inputs

- Raw defect description (free text, however rough)

Outputs

- Quality score out of 8 with per-dimension breakdown
- Missing field alerts (e.g. 'No reproduction steps', 'Severity not justified')
- Formatted JIRA bug report with: Summary, Description, Steps to Reproduce, Expected vs Actual, Environment, Severity, Priority, Attachments reminder
- Copy-to-clipboard button for direct JIRA paste

Scoring Dimensions

Dimension	What is checked
Title clarity	Is the summary specific enough to understand without reading the body?
Reproduction steps	Are numbered steps present?
Expected vs Actual	Are both explicitly stated?
Severity assignment	Is a severity level given (Critical/Major/Minor/Trivial)?
Priority assignment	Is priority set and does it align with severity?
Environment info	Browser, OS, app version mentioned?
Attachments noted	Screenshots, logs, video references included?
Defect uniqueness	Is it distinguishable from other defects?

Module 8 — SQL Test Data Generator

Purpose

Paste a CREATE TABLE statement and automatically generate test data rows covering all equivalence classes and boundary values, plus the SQL verification queries — bridging the gap between SQL training and test data design.

Inputs

- SQL CREATE TABLE statement (paste from DB schema)

Outputs

- Valid INSERT rows: one for each equivalence class (typical, min boundary, max boundary)
- Invalid INSERT rows: null on NOT NULL, oversized value, wrong type, FK violation hint
- SELECT verification queries: one per test scenario
- Export as .sql file

Behaviour Rules

14. Parser reads column names, types (INT, VARCHAR, DATE, DECIMAL), and constraints (NOT NULL, UNIQUE, DEFAULT, CHECK)
15. VARCHAR(n) generates: typical string, min-length 1 char, max-length n chars, n+1 chars (invalid)
16. INT columns generate: 0, 1, max-int, -1 (if no UNSIGNED constraint), NULL (if nullable)
17. DATE columns generate: today, past date, future date, invalid format

Module 9 — CI/CD Test Health Dashboard

Purpose

Paste Playwright's JSON test report output and instantly see a visual breakdown of pass/fail/flaky results, a prioritised fix queue, and a one-paragraph stand-up summary — turning an unreadable wall of JSON into something you can present in 30 seconds.

Inputs

- Playwright JSON reporter output (paste into textarea)
- Optional: previous run's JSON (for flaky test detection)

Outputs

- Summary stats: total tests, passed, failed, skipped, duration
- Results by test file and suite
- Flaky test list: tests that differ between two runs
- Fix priority queue: failures sorted by frequency
- One-paragraph stand-up summary (auto-generated text)

4. Export — The Glue Feature

The single most important non-module requirement. Every module contributes to a unified export.

Section in Export	Source Module	Content
Project Header	Shared context	Feature, version, tester, sprint, date
Test Cases	Case Formatter	All cases with type, priority, expected result, status
Checklist	Checklist Builder	All items with done/pending state
Coverage Summary	Heatmap	Feature areas with case counts
Boundary Test Values	Boundary Calculator	All boundary sets with valid/invalid flags
Equivalence Classes	Equivalence Partitioner	All partition sets
BDD Spec	Playwright Spec Builder	Feature file and step def skeleton
Defect Report	Defect DNA Analyzer	Formatted JIRA report
SQL Test Data	SQL Generator	INSERT and SELECT statements
CI Health Summary	CI Dashboard	Stand-up paragraph + failure list
RTM	Agile Sprint Planner	Story-to-test-case traceability matrix

Export formats: plain .txt (always available), .docx (Word document), copy to clipboard. Export is accessible from every tab via a persistent button in the top bar.

5. Non-Functional Requirements

5.1 Performance

- All module outputs must render within 500ms of user action
- No network requests required for core functionality
- Export generation must complete in under 2 seconds for up to 200 test cases

5.2 Usability

- Any module must be usable without reading documentation
- Every input field has a placeholder example
- Error states (empty input, invalid SQL) show a clear, non-technical message
- Works on laptop and desktop screens (min 1024px wide)

5.3 Compatibility

- Runs in Chrome, Firefox, and Edge (latest versions)
- No installation required — opens directly in the browser
- Exported .txt files are readable in any text editor

5.4 Reliability

- No data is sent to any server — everything stays local
- Corrupt or unexpected input (e.g. invalid SQL, malformed JSON) must not crash the app — show a graceful error message

6. Module Priority & Build Order

Recommended build sequence for a solo developer — start with the highest-value, lowest-complexity modules.

Phase	Module	Effort	Value	Why first
Phase 1	Shared Context + Case Formatter	Low	High	Core of the tool — everything depends on it
Phase 1	Checklist Builder	Low	High	Quick win, used before every release
Phase 1	Boundary Calculator	Low	High	Pure logic, no AI needed, immediate usefulness
Phase 1	Equivalence Partitioner	Low	High	Pure logic, pairs with Boundary Calculator
Phase 2	Coverage Heatmap	Medium	Medium	Visual layer on top of Phase 1 data
Phase 2	Export (all modules)	Medium	High	Makes everything usable professionally
Phase 2	Playwright Spec Builder	Medium	Very High	Daily use in automation work
Phase 3	Defect DNA Analyzer	Medium	High	Needs scoring logic but high daily value
Phase 3	SQL Test Data Generator	Medium	High	Needs SQL parser, medium complexity
Phase 3	CI/CD Test Health Dashboard	Medium	High	Needs JSON parser, Playwright report schema

7. Data Flow Between Modules

How modules connect

Data flows in one direction in v1.0 — source modules (Boundary, Equivalence, Sprint Planner, CI Dashboard) push output INTO the Case Formatter or Export. The Case Formatter is the central hub. The Export pulls from all modules at export time.

From Module	To Module	What flows	Trigger
Boundary Calculator	Case Formatter	One test case per boundary value	'Send to Cases' button
Equivalence Partitioner	Case Formatter	One test case per partition class	'Send to Cases' button
Case Formatter	Coverage Heatmap	Case count per feature area	Automatic on case add (v1.1)
Playwright Spec Builder	Export	Gherkin + step def text	Always included in export
Defect DNA Analyzer	Export	Formatted defect report	Always included in export
SQL Generator	Export	SQL INSERT + SELECT scripts	Always included in export
CI Dashboard	Export	Stand-up summary paragraph	Always included in export
All modules	Export document	Complete test plan	'Export Plan' button in top bar

8. Future Scope (v2.0 Ideas)

The following are out of scope for v1.0 but are natural extensions once the core is stable:

Feature	Description
JIRA API integration	Push defect reports and test cases directly to a JIRA project via API key
GitHub Actions support	Parse GitHub Actions YAML + Playwright report instead of Jenkins
AI-assisted locator suggestion	Use browser DOM screenshot to suggest real locators (Copilot-style)
Persistent storage	Save projects to browser localStorage or export/import as JSON
Team collaboration	Share a project link with read-only access
Test case versioning	Track changes to test cases across sprints
Playwright config generator	Generate playwright.config.ts based on project settings
GenAI prompt helper	Built-in prompts for ChatGPT/Copilot to explain failing tests or suggest fixes

9. Revision History

Version	Date	Author	Changes
1.0	March 2026	QA Trainee — VnV Jan 2026 Batch	Initial SRS — merged Test Planner Pro + 5 new module concepts

End of Document
QA Workspace Pro — SRS v1.0 | For questions or contributions, update revision history above.